

CS202: Programming Systems

Week 4 – Inheritance

10/2023

CS202 – What will be discussed?

- ☐ Introduction to inheritance
- ☐ Types of inheritance
- ☐ Derived class
 - Constructor
 - Destructor
 - Copy constructor & assignment operator
- ☐ Virtual functions and dynamic binding

Introduction

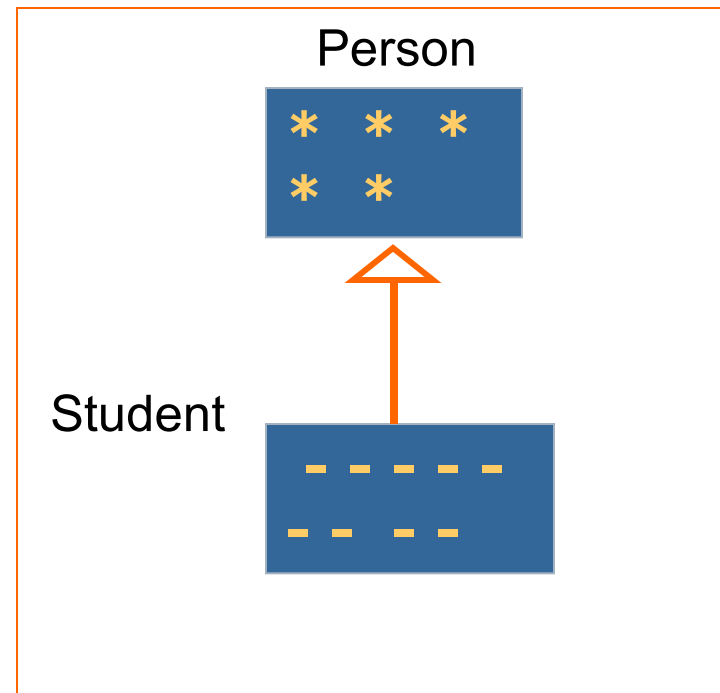
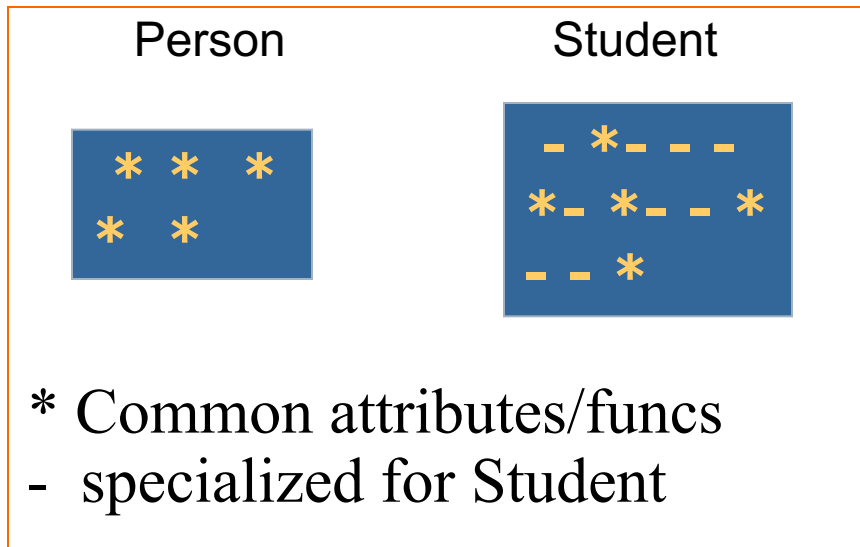
- A new concept does not come alone. When we introduce a class of **car** or **employee** for example. It may lead us to describe:
 - wheel, engine, driver etc.
 - Or: manager, director...
- To model them, we can use **class**. However, how can we model the relationship between them?

Introduction (cont.)

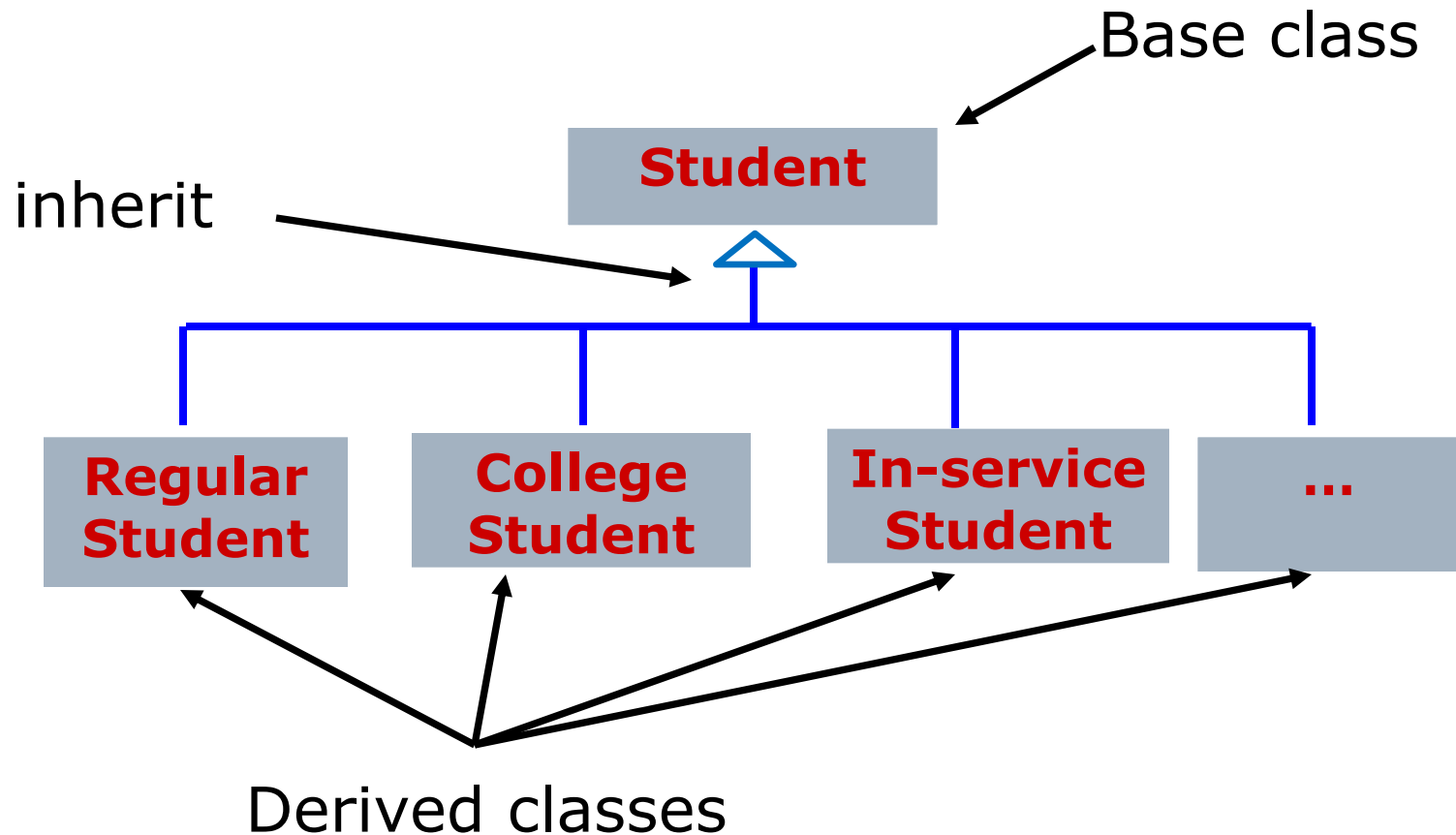
- ❑ In addition, re-usability of existing classes is one of the features of OOP.
- ❑ All of the features including attributes and behaviors of a class are also the features of another class.

Inheritance

□ Student “is a” person



An example



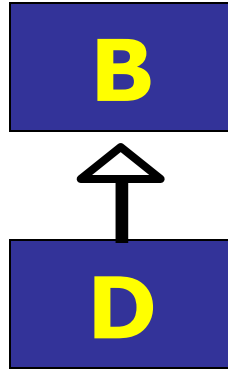
Syntax

□ In C++, the inheritance is described as:

```
class <Derived Class> : <TypeOfInheritance> <Base class>
{
    public:
        <public attributes/functions>

    private:
        <private attribute/functions>
};
```

Inheritance: notes



- ☐ All attributes/functions of B will be inherited to D
- ☐ **Private** of B is only accessible via **public** or **protected** of B

protected keyword

- **protected** of B is accessible from the derived class D but not from outside

```
class B
{
    public:
        ...
    protected:
        int t;
};
```

```
class D: public B
{
    public:
        void test()
        {
            cout << t;
        }
        ...
};
```

Types of inheritance in C++

There are 3 types:

- ☐ `public` inheritance (**IS-A** relationship)
- ☐ `protected`
- ☐ `private`

Notes: from now on, if there is no mention of what type of inheritance, it means public inheritance

Types of inheritance

- ❑ **public**: public and protected of the base class become public and protected of the derived class.
- ❑ **protected**: public and protected of the base class become protected of the derived class.
- ❑ **private**: public and protected of the base class become private of the derived class.

Inheritance: member functions

- Member functions of the base class are inherited in the derived class.
- What happens to:
 - Constructors
 - Destructors
 - Assignment operators
- Notes: private members or functions of the base class are inherited but ***only accessible via other public/protected functions*** of the base class

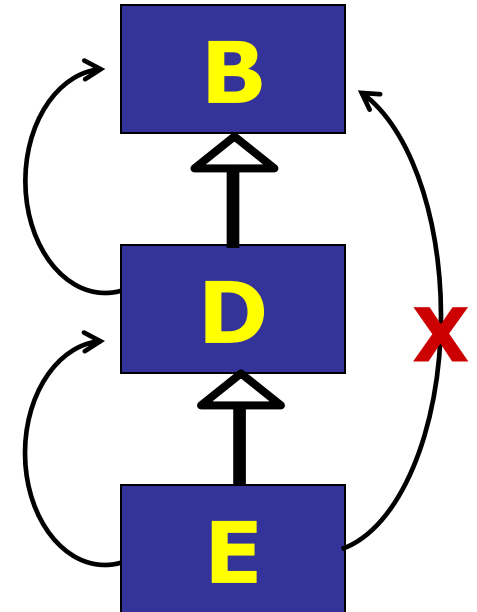
Constructors in inheritance

- When a new object of a derived class is created
 - The constructor of the base class is invoked first.
 - Then, the constructor of the derived class is invoked.
 - In the constructor of the derived class, we can specify which constructor of the base class is called. Otherwise, the default constructor of the base class will be invoked.

Constructors in inheritance

Notes:

- The constructor of the derived class is able to specify the constructor of the immediate base class to be called.



An example

```
class B
{
public:
    B();
    B(int);
};

class D : public B
{
public:
    D(int);
};
```

```
class B {
public:
    B();
    B(int);
};

class D : public B
{
public:
    D(int t) : B(t) {
        ...
    }
};
```

Destructor in inheritance

- When an object of the derived class finishes its lifespan:
 - The destructor of the derived class is invoked first.
 - Then, the destructor of the base class is called later.

“Re-define” member functions

- Sometimes, we need to “re-define” the member functions of the base class in the derived class.
 - It can be done by re-defining the functions inside the derived class

Notes: this re-definition will hide other overloading member functions of this function from the base class.

An example

```
class B {  
public:  
    void test();  
    void test(int);  
    void test(int, int);  
    ...  
};  
  
class D : public B  
{  
public:  
    void test(int);  
};
```

```
int main()  
{  
    D d;  
    int x, y;  
    ...  
    d.test(x); // OK  
    d.test(x, y); //error  
    ...  
}
```

using keyword

```
class B {  
public:  
    void test();  
    void test(int);  
    void test(int, int);  
    ...  
};  
  
class D : public B  
{  
public:  
    using B::test;  
    void test(int);  
};
```

```
int main()  
{  
    D d;  
    int x, y;  
    ...  
    d.test(x); // OK  
    d.test(x, y); //OK  
    ...  
}
```

Assignment operator

- To implement the assignment operator for the derived class:
 - Calling the assignment operator of the base class to assign data members of the base class part in the two objects first.
 - Then, implement the assignment for data member of the derived class part.

An example

```
D& D::operator=(const D& src)
{
    if (this == &src)
        return *this;
    B::operator=(src); //call the BASE
    delete [] ptr;
    iSize = src.iSize;
    ptr = new int [iSize];
    for (int i=0; i<iSize; ++i)
        ptr[i] = src.ptr[i];
    return *this;
}
```

